Лекция 4.
Файловые системы Linux и Windows

Цели лекции:

- Понять назначение файловой системы.
- Разобрать связь файла, имени, данных и служебной информации.
- Понять роль блоков, кластеров, журналирования и inode.
- Сравнить файловые системы Linux и Windows на базовом профессиональном уровне.
- Понимать монтирование, LVM и типовые ошибки администратора.

Список учебных вопросов:

1. Файл и файловая система: связь имени файла, данных на диске и служебной информации.
2. Блок или кластер: единица хранения данных и влияние размера блока.
3. Журналируемые и нежурналируемые файловые системы.
4. Файловые системы Linux: ext4, XFS, Btrfs.
5. VFS и inode: единый механизм работы с файловыми системами.
6. Единое дерево каталогов и монтирование в Linux.
7. Файловые системы Windows: FAT, FAT32 и NTFS.
8. Дополнительные возможности: LVM и альтернативные потоки данных NTFS.
9. Практическое значение файловых систем для администратора.

Основные термины лекции

- File system — файловая система, способ организации файлов на носителе.
- Block — блок, единица хранения данных в Linux и Unix-подобных системах.
- Cluster — кластер, единица выделения пространства в Windows.
- inode — индексный дескриптор файла в Linux.
- VFS — Virtual File System, виртуальная файловая система Linux.
- NTFS — New Technology File System, основная файловая система Windows.

1. Файл и файловая система

- Файл — именованная область данных, с которой работает пользователь или программа.
- Файл имеет имя, содержимое и служебные свойства.
- Файловая система определяет, как файлы размещаются на носителе.
- Она хранит структуру каталогов, права и метаданные.
- Без файловой системы раздел нельзя удобно использовать для хранения данных.

Что хранит файловая система

- Имена файлов и каталогов.
- Расположение данных на диске.
- Размер файла и время изменения.
- Права доступа и владельца.
- Служебную информацию для восстановления после сбоев.

Файл не равен имени файла

- Имя файла — это способ обратиться к данным.
- Данные файла находятся в блоках или кластерах на диске.
- Метаданные описывают свойства файла.
- Каталог связывает имя с записью файловой системы.
- Поэтому переименование файла не означает перемещение данных на диске.

Связь имени файла и данных

Файловая система хранит информацию о файлах (имена, размер, где находятся данные) и сами данные на носителе.

Имя файла — это человекочитаемая метка для доступа к данным.

Данные файла — это фактическое содержимое файла, хранящееся на диске.

1. ФАЙЛОВАЯ СИСТЕМА (метаданные)



Имя файла	Размер	Дата	Адреса данных на диске
Отчёт.docx	25 КБ	12.05.2024	9, 10, 11, 18
Таблица.xlsx	15 КБ	12.05.2024	20, 21, 22
Пейзаж.jpg	2 МБ	11.05.2024	40, 41, 42, ..., 55
Семья.png	1 МБ	11.05.2024	56, 57, 58, ..., 63
Песня.mp3	5 МБ	10.05.2024	100, 101, 102, ..., 120

2. ХРАНЕНИЕ ДАННЫХ НА ДИСКЕ



Данные файла могут храниться в разных местах диска (не обязательно подряд).
Файловая система знает, какие блоки принадлежат какому файлу.

3. КАК ЭТО РАБОТАЕТ



1. Пользователь открывает файл «Отчёт.docx»
2. ОС ищет имя файла в файловой системе (в метаданных)
3. Находит записи:
   - размер
   - дату
   - адреса данных
4. ОС читает данные из указанных блоков на диске
5. Данные собираются и передаются приложению

ВАЖНО ЗНАТЬ

- Имя файла и данные — это разные вещи. Имя — это указатель на данные.
- Файл может быть фрагментирован — его части хранятся в разных местах.
- При удалении файла удаляется запись в файловой системе, а место на диске помечается как свободное.

ПРИМЕР



Имя можно изменить — данные останутся теми же.
Данные можно потерять — имя останется «пустым».

ИТОГО: Файловая система — это «каталог», который связывает имена файлов с их данными на диске.
Когда вы работаете с файлом, ОС использует эту связь, чтобы найти и открыть нужные данные.

2. Блок или кластер

- Данные на диске хранятся порциями, а не как сплошной текст.
- В Linux часто говорят о блоках.
- В Windows часто используют термин кластер.
- Размер блока или кластера влияет на эффективность хранения.
- Слишком крупный размер может неэффективно расходовать место для маленьких файлов.

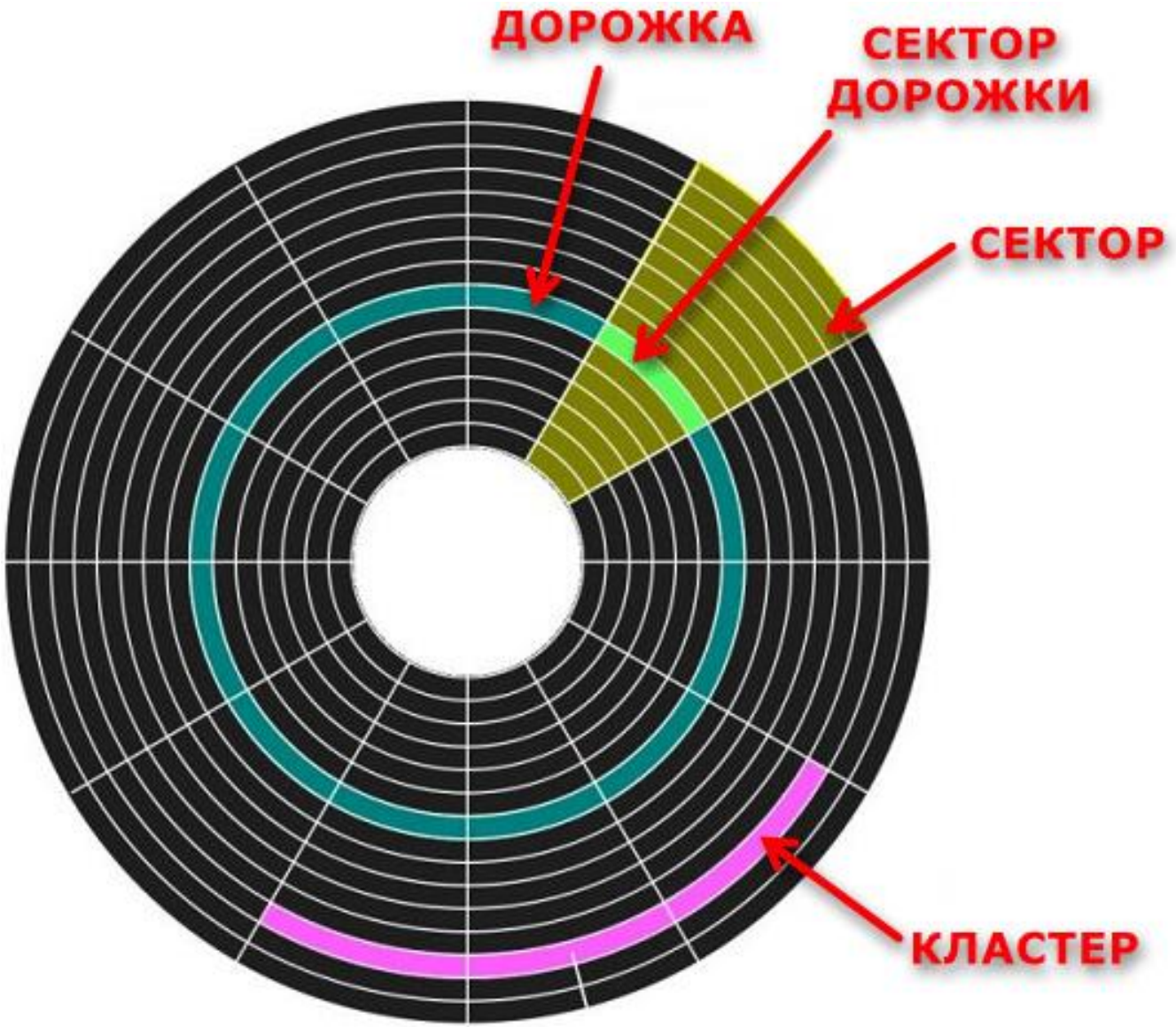
Блок в Linux

- Блок — минимальная единица хранения данных файловой системой.
- Файл может занимать один или много блоков.
- Размер блока выбирается при создании файловой системы.
- Блоки позволяют ОС управлять данными структурированно.
- Повреждение блоков может привести к потере части файла.

Кластер в Windows

- Кластер — единица выделения пространства в файловой системе Windows.
- Даже маленький файл занимает минимум один кластер.
- Размер кластера влияет на расход места и производительность.
- Для больших файлов крупный кластер может быть удобнее.
- Для множества маленьких файлов крупный кластер может приводить к потерям места.

Блоки и кластеры



Блоки и кластеры

Файловая система хранит данные на диске не побайтово,
а большими порциями — в блоках, а для выделения использует кластеры.

Блок — минимальная единица хранения данных на диске. Размер блока фиксирован и зависит от файловой системы (например, 4 КБ).

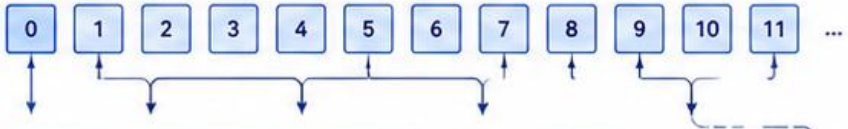
Кластер — группа из одного или нескольких блоков. Минимальное место, которое может быть выделено файлу.

1. КАК ЭТО УСТРОЕНО

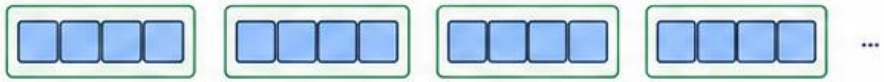
Диск



Блоки (пример: размер блока = 4 КБ)



Кластеры (пример: размер кластера = 16 КБ = 4 блока)



Кластер 0 | Кластер 1 | Кластер 2

Блоки — как маленькие ячейки на диске.
Они нумеруются последовательно.

Кластер — как «ячейка для хранения файла».
Файл занимает целое количество кластеров.

2. КАК ФАЙЛ ЗАНИМАЕТ МЕСТО

Пример: файл размером 20 КБ, размер кластера = 16 КБ (4 блока)

Файл 20 КБ



Занимает 2 кластера (32 КБ)



Файл использует 20 КБ, но на диске занято 32 КБ (весь второй кластер тоже выделен).

Потери места (фрагментация внутри кластера)

Если файл меньше размера кластера, оставшееся место в кластере не может быть использовано другими файлами.
Это приводит к внутренним потерям места.

3. ПРИМЕР: РАЗНЫЕ РАЗМЕРЫ КЛАСТЕРА

Размер кластера	Файл 10 КБ	Файл 20 КБ	Файл 70 КБ	Потери места
4 КБ (1 блок)	3 блока (12 КБ)	5 блоков (20 КБ)	18 блоков (72 КБ)	2 КБ
16 КБ (4 блока)	1 кластер (16 КБ)	2 кластера (32 КБ)	5 кластеров (80 КБ)	22 КБ
64 КБ (16 блоков)	1 кластер (64 КБ)	1 кластер (64 КБ)	2 кластера (128 КБ)	76 КБ

4. ВАЖНО ЗНАТЬ

- Размер кластера задаётся при форматировании раздела.
- Больше кластер — меньше служебной информация файловой системы, но больше потери места для маленьких файлов.
- Меньше кластер — меньше потери места, но больше служебной информации и может быть ниже производительность.



ИТОГО: Данные на диске хранятся и выделяются не побайтово, а целыми кластерами (группами блоков).
Это упрощает управление диском, но может приводить к потерям места, если файлы маленькие.

3. Журналируемые файловые системы

- Журналирование фиксирует намерения или изменения файловой системы.
- Журнал помогает восстановить согласованность после сбоя.
- Особенно важно при внезапном выключении питания.
- Журналирование снижает риск повреждения структуры файловой системы.
- Для серверов это критически важное свойство.

Нежурналируемые файловые системы

- Не ведут полноценный журнал изменений.
- Могут быть проще и легче.
- После сбоя могут требовать более долгой проверки.
- Риск повреждения структуры выше.
- Обычно не лучший выбор для важных серверных данных.

Что происходит при аварийном выключении

- Приложение могло записывать файл.
- Файловая система могла обновлять каталог или метаданные.
- Часть данных могла остаться в кэше.
- Без журнала структура может стать несогласованной.
- С журналом ОС быстрее понимает, какие операции нужно восстановить.

Журналирование файловой системы

Журналирование — это механизм, при котором файловая система сначала записывает информацию об изменениях в специальный журнал на диске, а затем применяет эти изменения к основной файловой системе. Это помогает защитить данные и файловую систему от повреждений при сбоях.

Журнал — область на диске, где последовательно записываются записи о планируемых изменениях.

Файловая система — основная структура на диске, где хранятся данные и метаданные (файлы, каталоги и т.д.).

1. КАК РАБОТАЕТ ЖУРНАЛИРОВАНИЕ



Журнал хранится на том же диске, что и файловая система (или на отдельном разделе/устройстве). Записи в журнале выполняются последовательно, что быстрее и безопаснее.

2. ЧТО ПРОИСХОДИТ ПРИ СБОЕ



3. ПРЕИМУЩЕСТВА ЖУРНАЛИРОВАНИЯ

- Защита от повреждения файловой системы при сбоях питания, зависаниях и ошибках.
- Быстрое восстановление после сбоев.
- Снижение необходимости в полной проверке диска.
- Повышение надёжности и целостности данных.
- Поддержка отката незавершённых операций.



4. ПРИМЕР ЗАПИСЕЙ В ЖУРНАЛЕ

№	Тип операции	Объект	Состояние	
1024	Создание файла	/docs/report.txt	Ожидает	Не применены к файловой системе
1025	Запись данных	/docs/report.txt	Ожидает	
1026	Обновление каталога	/docs/	Ожидает	
1024	Создание файла	/docs/report.txt	Выполнено	Уже применены к файловой системе
1025	Запись данных	/docs/report.txt	Выполнено	
1026	Обновление каталога	/docs/	Выполнено	

5. ГДЕ ИСПОЛЬЗУЕТСЯ

ext4 (Linux)	Режимы: data=ordered (по умолчанию), writeback, journal
NTFS (Windows)	Журнал USN и журнал транзакций (TxF) для метаданных
XFS (Linux)	Высокопроизводительное журналирование метаданных
Btrfs (Linux)	Copy-on-Write + журналирование метаданных

ИТОГО: Журналирование значительно повышает надёжность файловой системы. Хотя оно создаёт небольшой дополнительный накладной расход на запись, выигрыш в целостности данных и восстановлении после сбоев обычно гораздо важнее.

4. Файловые системы Linux

- Linux поддерживает разные файловые системы.
- Выбор зависит от задачи, нагрузки и требований к надежности.
- На первом курсе важно знать ext4, XFS и Btrfs на уровне назначения.
- Администратор должен уметь определить тип файловой системы.
- Нельзя форматировать раздел без понимания, что будет удалено.

ext4

- ext4 — распространенная файловая система Linux.
- Подходит для рабочих станций и многих серверных задач.
- Поддерживает журналирование.
- Хорошо документирована и широко используется.
- Часто является безопасным выбором для учебных Linux ВМ.

XFS

- XFS — производительная журналируемая файловая система.
- Хорошо подходит для больших файлов и серверных нагрузок.
- Часто используется в корпоративных Linux-системах.
- Не всегда удобна для уменьшения размера раздела.
- Требует понимания сценария использования.

Btrfs

- Btrfs — современная файловая система с расширенными возможностями.
- Поддерживает снапшоты на уровне файловой системы.
- Может использовать контроль целостности данных.
- Имеет больше возможностей, но требует аккуратного администрирования.
- Для первого курса достаточно обзорного понимания.

ФАЙЛОВЫЕ СИСТЕМЫ LINUX

Файловая система определяет, как данные хранятся, организуются и извлекаются на диске.

Надёжность защита данных и целостность

Производительность быстрая работа с файлами

Гибкость подходит для разных задач и устройств

	ext4	XFS	Btrfs	ZFS	F2FS
Описание	Надёжная и проверенная файловая система по умолчанию во многих дистрибутивах.	Высокопроизводительная ФС для больших файлов и параллельных нагрузок.	Современная ФС с множеством встроенных возможностей.	Мощная ФС и менеджер томов с акцентом на надёжность.	Файловая система, оптимизированная для флеш-памяти (SSD, eMMC, UFS).
Преимущества	• Стабильность • Журналирование • Поддержка больших файлов и томов	• Отличная производительность • Масштабируемость • Поддержка больших томов	• Снимки (snapshots) • Сжатие данных • Проверка целостности • Гибкие подтома	• Проверка целостности данных • Снимки и клоны • Сжатие • Отличная защита от потерь данных	• Оптимизация для SSD • Меньше износа флеш-памяти • Высокая скорость
Недостатки	• Медленнее современных ФС в некоторых сценариях	• Нет сжатия «из коробки»	• Сложнее в настройке • Может быть медленнее в некоторых задачах	• Высокие требования к памяти (RAM) • Сложнее в управлении	• Не подходит для обычных HDD
Когда использовать	• Повседневные задачи, серверы, общие цели	• Серверы, большие файлы, высокие нагрузки	• Рабочие станции, домашние системы, нужны снимки и гибкость	• Хранилища данных, серверы, важные данные	• Устройства с флеш-памятью: SSD, смартфоны, встраиваемые системы

ИТОГО

ext4 — универсальный выбор для большинства пользователей.

XFS — для высокой производительности и масштабируемости.

Btrfs — для гибкости, снимков и защиты данных.

ZFS — для максимальной надёжности и хранения важных данных.

F2FS — для устройств с флеш-памятью и SSD.

5. VFS и inode

- VFS — Virtual File System, слой абстракции в Linux.
- Он позволяет Linux работать с разными файловыми системами через единый интерфейс.
- Благодаря VFS программы одинаково открывают файлы на разных ФС.
- inode хранит служебную информацию о файле.
- Имя файла и inode — разные элементы файловой системы.

inode

- inode — структура с метаданными файла.
- Хранит владельца, права, размер и ссылки на блоки данных.
- Обычно не хранит само имя файла.
- Каталог связывает имя файла с номером inode.
- Нехватка inode может помешать создать файл даже при свободном месте.

VFS

- VFS скрывает различия ext4, XFS, Btrfs и других ФС.
- Приложение вызывает стандартные операции: открыть, прочитать, записать.
- Ядро через VFS передает запрос конкретной файловой системе.
- Это упрощает работу программ и администрирование.
- VFS объясняет, почему Linux может монтировать разные типы ФС.

VFS и inode: как работает файловая система

VFS (Virtual File System) — это универсальный уровень абстракции в ядре ОС,
а inode — структура, в которой хранятся метаданные файла в файловых системах Linux/Unix.

VFS
Виртуальный интерфейс, который предоставляет единый API для работы с разными файловыми системами.

inode
Структура данных на диске, которая содержит метаданные файла (не его имя и не данные).

1. АРХИТЕКТУРА: VFS КАК УНИВЕРСАЛЬНЫЙ СЛОЙ

Приложения
- Терминал
- Текстовый редактор
- Веб-сервер
- ...

→ **Системные вызовы**
open()
read()
write()
stat()
...

→ **VFS (Virtual File System)**
- Единый интерфейс для файловых систем
- Работа с inodes, dentry, файлами и кэшами
- Независимость приложений от конкретной ФС

Пути и каталоги (dentry) | Иноды (inode) | Файловые объекты (file) | Кэш и буферы

Драйверы файловых систем
ext4 | XFS | Btrfs | NTFS | FAT32 | ...

Устройства хранения (диски, SSD, RAID, сетевые хранилища)

Что даёт VFS
- Единый API для приложений
- Лёгкое добавление новых ФС
- Кэширование и управление памятью
- Безопасность и права доступа

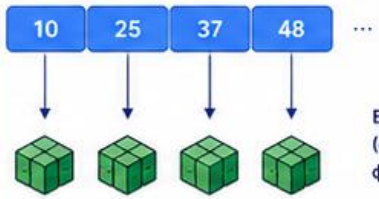
2. ЧТО ТАКОЕ INODE

inode (index node) — это структура данных, в которой хранятся все метаданные файла, кроме его имени и самих данных.

Пример inode (ext4)

Тип файла	Обычный файл
Права доступа	rwxr-xr--
Владелец (UID)	1000
Группа (GID)	1000
Размер файла	123456 байт
Время доступа (atime)	12.05.2024 10:15
Время изменения (mtime)	12.05.2024 10:15
Время изменения метаданных (ctime)	12.05.2024 10:20
Счётчик ссылок (hard links)	2
Указатели на блоки с данными	10 25 37 ...
Дополнительные атрибуты	...

Как inode связан с данными



- Имя файла хранится в каталоге и указывает на номер inode.
- Несколько имён (жёсткие ссылки) могут указывать на один и тот же inode.

3. КАК ОТКРЫВАЕТСЯ ФАЙЛ (пример)

/home/user/doc.txt

1. Приложение запрашивает открытие файла по пути
→ VFS разбирает путь и ищет в кэше dentry
→ inode #12345 — Находит inode файла
→ Проверяет права доступа
→ Во:вращает файловый дескриптор приложению

4. INODE: ВАЖНЫЕ ОСОБЕННОСТИ

- На каждый файл (и каталог) — свой inode.
- inode не содержит имя файла.
- Файл может иметь несколько имён (жёсткие ссылки).
- Каталог — это специальный файл, который хранит имена и номера inodes.
- Удаление имени не удаляет данные, пока есть другие ссылки на inode.

Пример: жёсткие ссылки



Счётчик ссылок в inode: 2

Обе записи в каталоге указывают на один и тот же inode. Данные удалятся только когда счётчик ссылок станет 0.

5. ПРИМЕР: EXT4

Каталог (имена файлов)

doc.txt	inode #1001
photo.jpg	inode #1002
notes.txt	inode #1003
...	...

→ inode #1001 (метаданные) → Блоки данных на диске

ИТОГО
- VFS предоставляет единый интерфейс для работы с разными файловыми системами.
- inode хранит метаданные файла и ссылки на его данные, но не имя.
- имена файлов хранятся в каталогах и указывают на соответствующие inodes.
- Такая архитектура обеспечивает гибкость, надёжность и эффективность работы с данными.

6. Единое дерево каталогов и монтирование

- В Linux нет букв дисков C: и D: как в Windows.
- Есть единое дерево каталогов с корнем /.
- Раздел или диск подключается к этому дереву через точку монтирования.
- Точка монтирования — каталог, через который доступен раздел.
- Это важно понимать при работе с серверными дисками.

Точка монтирования

- Точка монтирования выглядит как обычный каталог.
- Например, /mnt/data или /var/log.
- После монтирования содержимое раздела доступно через этот каталог.
- Если раздел не смонтирован, данные могут быть недоступны.
- Ошибка монтирования может нарушить работу службы.

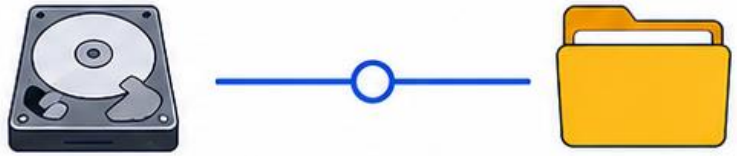
Постоянное монтирование

- Для постоянного подключения используют файл /etc/fstab.
- В нем указывают устройство, точку монтирования и параметры.
- Ошибка в /etc/fstab может помешать загрузке системы.
- Лучше использовать UUID, а не случайное имя диска.
- Перед изменением fstab нужен снапшот или резервная копия.

Монтирование в Linux

Как раздел или устройство становится доступным в файловой системе

1 Что такое монтирование?



Монтирование — это подключение файловой системы устройства к общей файловой иерархии Linux.

2 Основные элементы

Устройство
/dev/sdb1

Файловая система
ext4, xfs, vfat

Точка монтирования
/mnt/usb
или /home

Дерево каталогов
всё доступно внутри единой структуры /

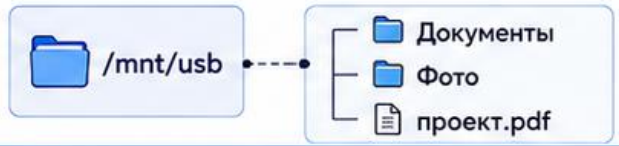
3 Как это работает



4 Пример



Устройство	Команда	Результат
/dev/sdb1	mount /dev/sdb1 /mnt/usb	Файлы становятся доступны в /mnt/usb



5 Полезные команды

lsblk
показать диски и разделы

mount
подключить или показать монтирования

umount
отключить

df -h
показать занятое место

cat /etc/fstab
автоподключение

6 Что важно помнить

В Linux всё подключается в одно дерево каталогов

Без точки монтирования доступ к файлам не появится

umount нужно делать перед безопасным извлечением носителя

/etc/fstab
используется для автоматического монтирования

Главная идея: монтирование связывает устройство и каталог, чтобы данные стали доступны в Linux.

7. Файловые системы Windows

- Windows использует буквы дисков и томов.
- Пользователь часто видит C:, D:, E:.
- Основная современная файловая система Windows — NTFS.
- FAT и FAT32 встречаются на съемных носителях и старых системах.
- Выбор файловой системы влияет на размер файлов, права и надежность.

FAT и FAT32

- FAT — File Allocation Table, старая файловая система.
- FAT32 — более поздний вариант FAT.
- Хорошо совместима с разными устройствами.
- Имеет ограничения по размеру файла и возможностям безопасности.
- Не подходит как основная ФС для современного Windows Server.

NTFS

- NTFS — New Technology File System.
- Поддерживает права доступа и владельцев.
- Поддерживает журналирование.
- Используется как основная файловая система Windows.
- Важна для файловых серверов, профилей пользователей и системных дисков.

Права NTFS

- Права NTFS управляют доступом к файлам и каталогам.
- Можно разрешить чтение, изменение, выполнение или полный доступ.
- Права могут назначаться пользователям и группам.
- Share-права и NTFS-права работают вместе для сетевых ресурсов.
- Ошибка прав — частая причина проблем доступа.

ФАЙЛОВЫЕ СИСТЕМЫ WINDOWS

Файловая система определяет, как данные хранятся, организуются и извлекаются на диске.

Надёжность защита данных | Производительность быстрая работа | Совместимость с устройствами

	FAT32	exFAT	NTFS	ReFS	CDFS / UDF
Описание	Старая, но широко совместимая файловая система.	Современная ФС для флешек и карт памяти большого объёма.	Основная ФС Windows для внутренних дисков.	Современная ФС для надёжности и работы с большими объёмами данных.	Файловые системы для оптических дисков.
Преимущества	• Очень совместима с разными устройствами • Работает почти везде • Простая и надёжная	• Поддержка больших файлов (>4 ГБ) • Оптимизирована для флешек • Широкая совместимость	• Поддержка больших файлов и томов • Журналирование и защита данных • Разрешения и шифрование • Сжатие и квоты	• Высокая целостность данных • Автоматическое исправление ошибок • Хорошо работает с большими объёмами	• Запись и чтение оптических дисков • Поддержка больших файлов • Совместимость с разными форматами дисков
Недостатки	• Файл не более 4 ГБ • Раздел не более 2 ТБ • Нет безопасности и журналирования	• Нет журналирования и прав доступа • Меньше возможностей безопасности	• Чуть ниже производительность на старых устройствах • Не все системы поддерживают запись	• Не для системного диска • Ограниченная совместимость со старыми версиями Windows	• Низкая скорость по сравнению с современными дисками • Ограничено перезаписью
Когда использовать	Флешки и карты памяти небольшого объёма, старые устройства.	Флешки и карты памяти большого объёма, обмен файлами между устройствами.	Внутренние диски Windows, рабочие файлы, программы, системные диски.	Серверы, хранилища данных, резервные копии, критически важные данные.	CD, DVD, Blu-ray диски, архивные данные.

ИТОГО

- FAT32 — для максимальной совместимости и небольших объёмов.
- exFAT — для флешек большого объёма, и обмена файлами.
- NTFS — лучшая для Windows: безопасность, надёжность и все возможности.
- ReFS — для надёжного хранения больших объёмов данных.
- CDFS / UDF — для оптических дисков и архивных данных.

8. LVM и альтернативные потоки NTFS

- LVM — Logical Volume Manager, менеджер логических томов Linux.
- Позволяет гибко управлять томами поверх дисков и разделов.
- Можно расширять логические тома при добавлении дисков.
- Для первого курса достаточно обзорного понимания.
- Глубокая настройка LVM изучается позже.

Альтернативные потоки данных NTFS

- Alternate Data Streams — альтернативные потоки данных NTFS.
- Позволяют хранить дополнительные данные внутри файла.
- Пользователь обычно не видит их в обычном просмотре.
- Могут использоваться системами безопасности и приложениями.
- Для данном этапе важно знать, что такая возможность существует.

LVM и обычные разделы: в чём разница?

И LVM, и обычные разделы используются для организации хранения данных на диске.
Но подход к управлению пространством и гибкость у них сильно отличаются.

Обычные разделы — независимые разделы фиксированного размера на диске. Изменить их размер сложно и обычно требует резервного копирования.

LVM (Logical Volume Manager) — менеджер логических томов в Linux, который добавляет уровень абстракции и даёт гибкость в управлении хранилищем.

1. ОБЫЧНЫЕ РАЗДЕЛЫ

Как устроено

Каждый раздел — это отдельная область на диске.
Файловая система создаётся прямо на разделе.





Особенности

- Размер фиксируется при создании.
- Чтобы изменить размер — нужно резервное копирование и сложные операции (или пересоздание).
- Каждый раздел независим.
- Простая структура, не требует дополнительного ПО.

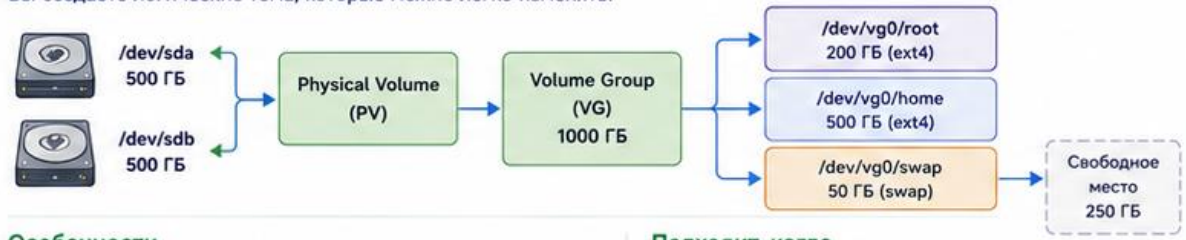
Подходит, когда

- Нужна простота и стабильность.
- Конфигурация не меняется.
- Сервер или ПК с предсказуемой нагрузкой.

2. LVM (LOGICAL VOLUME MANAGER)

Как устроено

LVM добавляет слой абстракции между физическими дисками и файловыми системами.
Вы создаёте логические тома, которые можно легко изменять.



Особенности

- Логические тома можно создавать, удалять, расширять и сжимать «на лету».
- Легко использовать несколько дисков как единое хранилище.
- Поддержка снимков (snapshot), зеркалирования, RAID (на уровне LVM).
- Требует немного больше ресурсов и знаний.

Подходит, когда

- Нужна гибкость и масштабируемость.
- Планируется рост данных.
- Важно легко управлять пространством без простоев.

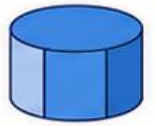
3. СРАВНЕНИЕ

Критерий	Обычные разделы	LVM
Гибкость размера	✗ Изменить размер сложно	✓ Легко изменять размер томов
Управление дисками	✗ Каждый раздел на одном диске	✓ Можно объединять несколько дисков в пул
Снимки (snapshot)	✗ Нет (только с помощью LVM или сторонних инструментов)	✓ Есть встроенная поддержка
Отказоустойчивость	− Зависит от настроек (RAID и т.д.)	✓ Есть возможности (зеркала, RAID через LVM)
Сложность настройки	✓ Просто	− Сложнее (нужно понимание LVM)
Производительность	✓ Немного выше (меньше слой абстракции)	− Незначительное снижение (обычно не заметно)
Рекомендуется для	Простых систем, небольших серверов, фиксированных задач	Серверов, систем с ростом данных, виртуализации, продакшн-сред

4. ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

Обычные разделы

- Домашний ПК
- Небольшой сервер с фиксированными требованиями
- Системы, где важна максимальная простота



LVM

- Серверы с растущими объёмами данных
- Виртуализация (KVM, VMware, Proxmox)
- Базы данных, файловые хранилища
- Когда нужна высокая доступность и гибкость



ИТОГО Обычные разделы — это просто и надёжно, но негибко.
LVM — это гибкость, масштабируемость и дополнительные возможности, но требует чуть больше знаний и ресурсов.

9. Практическое значение для администратора

- Файловая система влияет на надежность и возможности хранения.
- Администратор выбирает ФС под ОС и задачу.
- Нужно понимать, где находятся данные и как они подключены.
- Нужно уметь диагностировать ошибки монтирования и прав.
- Перед форматированием или изменением ФС нужен снапшот.

Типовые ошибки

- Форматировать не тот раздел.
- Монтировать раздел не в ту точку.
- Не прописать постоянное монтирование.
- Перепутать свободное место и количество inode.
- Назначить неверные права NTFS или Linux.
- Считать журналирование заменой резервного копирования.

Итоги лекции

- Файловая система связывает имена, данные и служебную информацию.
- Блоки и кластеры определяют, как выделяется место на диске.
- Журналирование помогает восстановиться после сбоя.
- Linux использует единое дерево каталогов и точки монтирования.
- Windows использует тома с буквами и NTFS как основную ФС.
- Администратор должен понимать ФС перед форматированием и назначением прав.

Контрольные вопросы

1. Для чего операционной системе нужна файловая система?
2. Что такое блок или кластер в контексте хранения данных?
3. Почему журналирование важно для серверных файловых систем?
4. Чем ext4, XFS и Btrfs отличаются по назначению?
5. Что такое inode и какую информацию он хранит?
6. Почему в Linux используется единое дерево каталогов?
7. Для чего используется точка монтирования?
8. Почему NTFS является основной файловой системой Windows?
9. Какие ошибки могут возникнуть при форматировании, монтировании или назначении прав?